

You're absolutely right! Multiple `Include()` statements can cause performance issues. Here are the problems and better alternatives:

Problems with Multiple Includes

1. Cartesian Product Problem

```
// BAD: Creates huge result set
var query = _context.Employees
    .Include(e => e.Department)      // Joins Department
    .Include(e => e.Projects)        // Joins Projects
    .Include(e => e.Skills)          // Joins Skills
    .Include(e => e.Manager)         // Joins Manager
    .ToListAsync();
```

This creates a massive Cartesian product:

100 employees × 5 departments × 10 projects × 8 skills × 3 managers = **120,000 rows!**

2. Duplicate Data Transfer

```
// Each employee row contains ALL related data
Employee: John, Dept: IT, Project: Web, Skill: C#, Manager: Alice
Employee: John, Dept: IT, Project: Web, Skill: SQL, Manager: Alice
Employee: John, Dept: IT, Project: Web, Skill: JS, Manager: Alice
// ... many duplicates
```

Better Solutions

Solution 1: Split Queries (EF Core 5+)

```
// GOOD: Separate queries for each relationship
var query = _context.Employees
    .Include(e => e.Department)
    .Include(e => e.Projects)
```

```
.Include(e => e.Skills)
.AsSplitQuery() // <-- Key improvement
.ToListAsync();
```

Result: 4 separate queries instead of 1 massive query:

```
-- Query 1: Employees + Departments
SELECT e.*, d.* FROM Employees e
LEFT JOIN Departments d ON e.Did = d.Did

-- Query 2: Employees + Projects
SELECT e.*, p.* FROM Employees e
LEFT JOIN Projects p ON e.Eid = p.EmployeeId

-- Query 3: Employees + Skills
SELECT e.*, s.* FROM Employees e
LEFT JOIN Skills s ON e.Eid = s.EmployeeId
```

Solution 2: Select Only What You Need

```
// GOOD: Project specific data
var employees = await _context.Employees
    .Select(e => new EmployeeDTO
    {
        Eid = e.Eid,
        FirstName = e.FirstName,
        LastName = e.LastName,
        DepartmentName = e.Department.DName,
        ProjectCount = e.Projects.Count,
        SkillNames = e.Skills.Select(s => s.Name).ToList()
    })
    .AsNoTracking()
    .ToListAsync();
```

Solution 3: Lazy Loading with Explicit Control

```
// GOOD: Load base data first, then load related data as needed
public async Task<EmployeeDetailsDTO> GetEmployeeDetails(int id)
{
    var employee = await _context.Employees
        .AsNoTracking()
        .FirstOrDefaultAsync(e => e.Eid == id);

    if (employee == null) return null;

    // Load related data separately
    var department = await _context.Departments
        .AsNoTracking()
        .FirstOrDefaultAsync(d => d.Did == employee.Did);

    var projects = await _context.Projects
        .AsNoTracking()
        .Where(p => p.EmployeeId == employee.Eid)
        .ToListAsync();

    return new EmployeeDetailsDTO
    {
        Employee = employee,
        Department = department,
        Projects = projects
    };
}
```

Solution 4: Eager Loading with ThenInclude

```
// BETTER: More precise loading
var employees = await _context.Employees
    .Include(e => e.Department)
        .ThenInclude(d => d.Manager) // Load Department's Manager
    .Include(e => e.Projects)
        .ThenInclude(p => p.Client) // Load Project's Client
    .AsNoTracking()
```

```
.ToListAsync();
```

Performance Comparison

Multiple Includes (Bad)

```
// 1 query, 50,000 rows returned, 2MB data transfer
var result = await _context.Employees
    .Include(e => e.Department)
    .Include(e => e.Projects)
    .Include(e => e.Skills)
    .ToListAsync();
```

Split Queries (Good)

```
// 4 queries, 200 rows total, 50KB data transfer
var result = await _context.Employees
    .Include(e => e.Department)
    .Include(e => e.Projects)
    .Include(e => e.Skills)
    .AsSplitQuery()
    .ToListAsync();
```

Select Projection (Best)

```
// 1 query, 100 rows, 20KB data transfer
var result = await _context.Employees
    .Select(e => new EmployeeDTO { /* only needed fields */ })
    .ToListAsync();
```

Best Practices for Your Project

1. Use Split Queries for Complex Includes

```
// In your repository
public async Task<IEnumerable<NETEmployee>>
GetEmployeesWithAllDetails()
{
    return await _context.Employees
        .Include(e => e.Department)
        .AsSplitQuery() // Enable split queries
        .AsNoTracking()
        .ToListAsync();
}
```

2. Use Select for API Responses

```
// For API endpoints - return only what client needs
public async Task<IEnumerable<EmployeeListDTO>> GetEmployeesForList()
{
    return await _context.Employees
        .Select(e => new EmployeeListDTO
        {
            Eid = e.Eid,
            FullName = e.FirstName + " " + e.LastName,
            Email = e.Email,
            DepartmentName = e.Department.DName
        })
        .AsNoTracking()
        .ToListAsync();
}
```

3. Configure Split Queries Globally

```
// In Program.cs or DbContext
builder.Services.AddDbContext<OrganizationDbContext>(options =>
{
    // ...
})
```

```

        options.UseSqlServer(connectionString)
            .UseQueryTrackingBehavior(QueryTrackingBehavior.NoTracking)
            .EnableSensitiveDataLogging(false);
    });

// Or in DbContext
protected override void OnConfiguring(DbContextOptionsBuilder
optionsBuilder)
{
    optionsBuilder.UseSqlServer(connectionString)
        .UseQuerySplittingBehavior(QuerySplittingBehavior.Sp
litQuery);
}

```

When to Use Each Approach

Scenario	Best Approach
Simple API list	Select() projection
Complex object graph	AsSplitQuery()
Admin dashboard	Multiple specific queries
Reporting	Raw SQL or views
Mobile app	Minimal DTOs with Select()

The key is to **avoid loading unnecessary data** and **prevent Cartesian products** that can kill performance.

Summary

Problems with Multiple Includes:

Cartesian product - Massive result sets

Duplicate data - Same data repeated many times

Poor performance - Slow queries, high memory usage

Better Solutions:

AsSplitQuery() - Separate queries for each include

Select() projection - Only load needed fields

Explicit multiple queries - Control exactly what loads

ThenInclude() - More precise nested loading

Recommendation for your project:

Use **AsSplitQuery()** for complex includes

Use **Select()** projections for API responses

Keep **AsNoTracking()** for read operations

This approach will significantly improve your API performance, especially with pagination and large datasets.